

Práctica 1 — Debugging y Prueba de Código

Reparación Automática de Software — 2026

Universidad Nacional de Río Cuarto (UNRC)

Resumen. Este informe resuelve los cuatro ejercicios de la Práctica 1. Los Ejercicios 1 y 2 documentan la instalación de *The Debugging Book* y los conceptos centrales de sus capítulos *01_Intro* y *02_Observing*. Los Ejercicios 3 y 4 analizan los módulos `max.py` y `person.py`: para cada test que falla se identifica el **defecto**, el **error** y la **falla**, y se propone la corrección. **Todos los resultados de los tests fueron verificados ejecutando el código** (Python 3.9).

Vocabulario base (Cap. 1)

Los tres términos que se usan en todo el informe (del capítulo *Introduction to Debugging*):

- **Defecto (fault):** la porción de código escrita incorrectamente (la causa raíz en el texto del programa).
- **Error (infección):** un estado interno del programa durante la ejecución que es incorrecto por culpa del defecto (el texto lo llama *infection*). Un defecto puede no producir error si nunca se ejecuta o el estado malo no llega a propagarse.
- **Falla (failure):** el comportamiento observable e incorrecto (salida errónea, excepción, resultado equivocado) que percibe quien usa el programa.

La cadena causal es: **Defecto** → **Error** → **Falla**.

1. Instalación de *The Debugging Book*

Pasos ejecutados sobre Ubuntu 22.04+ (x86-64), siguiendo la guía:

1. Descargar desde debuggingbook.org el paquete de notebooks: *resources* → *All Notebooks* (.zip).
2. Descomprimir y aplicar los ajustes previos (Graphviz + fix de `requirements.txt`).

```
# 1. Descomprimir el material
unzip debuggingbook-notebooks.zip
cd debuggingbook-notebooks      # carpeta resultante

# 2. Fix requerido por la guía (punto d): en requirements.txt,
# la línea 30 debe empezar con "showast2" en lugar de "showast".
sed -i '30s/^showast/showast2/' requirements.txt

# 3. Asegurar Graphviz (necesario para los grafos de las notebooks)
dot --version || sudo apt install graphviz

# 4. Crear y activar un entorno virtual aislado
python3 -m venv .venv
source .venv/bin/activate

# 5. Instalar dependencias (OJO: la guía dice "requirements"
# pero el comando correcto lleva la extensión .txt)
pip install -r requirements.txt

# 6. Levantar las notebooks
jupyter notebook
# Si no abre el navegador: http://localhost:8888/tree
```

Notas / correcciones respecto del enunciado:

- El fix del punto (d) evita el conflicto de nombres del paquete `showast` por `showast2`.
- El punto (h) dice `pip install -r requirements`; el comando correcto es `pip install -r requirements.txt`.
- Conviene crear el `venv` dentro de la carpeta descomprimida para mantener todo aislado.

2. Capítulos *01_Intro* y *02_Observing*

01 — Introduction to Debugging

Ideas centrales completadas:

- Distinción **defecto** / **error** / **falla** (ver vocabulario base). Es el marco que se usa en los Ejercicios 3 y 4.
- **El método científico del debugging**: observar la falla, formular una *hipótesis* sobre la causa, predecir una consecuencia observable, experimentar para confirmarla o refutarla, y repetir hasta aislar el defecto. Es un proceso iterativo, no adivinanza.
- **Debugging = búsqueda de la causa raíz**, no sólo enmascarar el síntoma: corregir el defecto, no la falla.

- Ejemplo canónico del libro: `remove_html_markup`, donde un caso con comillas rompe la función y sirve para practicar la formulación de hipótesis.

02 — How to Observe

Técnicas para observar el estado del programa y localizar el error:

- **Observación con `print` / `logging`:** rápida pero invasiva; útil para un primer sondeo.
- **Depurador interactivo** (`pdb`, breakpoints): inspeccionar variables, avanzar paso a paso, examinar el *stack*.
- **Aserciones** (`assert`): codifican invariantes; detectan el error *cerca* de su origen, antes de que se propague hasta la falla.
- El objetivo es acortar la distancia entre *defecto* y *error observado*: cuanto antes se detecta el estado infectado, más fácil es ubicar la línea culpable.

3. Módulo `max.py`

El módulo tiene dos funciones. `max_int` es correcta; el defecto está en `max_int_seq`.

3.1 Tests

Archivo entregable: `test_max.py` (ejecutable con `pytest -v`).

```

1  from max import max_int, max_int_seq
2
3  # --- max_int: correcta, todos pasan ---
4  def test_max_int_a_mayor():    assert max_int(5, 3) == 5    # PASA
5  def test_max_int_b_mayor():    assert max_int(3, 5) == 5    # PASA
6  def test_max_int_iguales():    assert max_int(4, 4) == 4    # PASA
7
8  # --- max_int_seq ---
9  # Spec asumida (segun el codigo): distinta longitud -> gana la mas larga;
10 # igual longitud -> gana la que tiene mas maximos elemento-a-elemento.
11 def test_seq_distinta_longitud():
12     assert max_int_seq([1], [2, 3]) == [2, 3]                # PASA
13 def test_seq_igual_longitud_gana_b():
14     assert max_int_seq([1, 2], [3, 4]) == [3, 4]              # PASA (por
15     coincidencia)
16 def test_seq_igual_longitud_gana_a():
17     assert max_int_seq([9, 9], [1, 1]) == [9, 9]              # FALLA
18 def test_seq_igual_longitud_gana_a_2():
19     assert max_int_seq([3, 5], [1, 2]) == [3, 5]              # FALLA

```

Resultado verificado: los tests de `max_int` y el de distinta longitud [**PASA**]. El caso en que `b` gana legítimamente [**PASA**] *por coincidencia* (ver abajo). Los dos últimos [**FALLA**]: para `[9,9]` vs `[1,1]` la función devuelve `[1,1]` cuando debería devolver `[9,9]`.

3.2 Defecto, error y falla

Defecto (fault): en la línea 28, la comparación es `if max_int(a[i], b[i]) == a:` — compara el máximo (un *int*) contra la *lista* completa `a`. Debería comparar contra el elemento `a[i]`.

Error (infección): `max_int(a[i], b[i])` devuelve un `int`, que nunca es igual a la lista `a`; por lo tanto la condición es *siempre falsa*. El contador `max_a` queda en `0` y sólo se incrementa `max_b`, aunque `a` tenga todos los máximos. El estado interno de los contadores es incorrecto.

Falla (failure): en el caso de igual longitud la función *siempre* devuelve `b`. Con `max_int_seq([9,9],[1,1])` retorna `[1,1]` en lugar de `[9,9]`.

Coincidencia: el test con `[1,2]` vs `[3,4]` pasa porque ahí `b` efectivamente debía ganar; el bug queda oculto (*coincidental correctness*).

3.3 Corrección

```

1      for i in range(0, len(a)):
2          if max_int(a[i], b[i]) == a[i]:      # FIX: a[i] en lugar de a
3              max_a += 1
4          else:
5              max_b += 1

```

Con este cambio, los cuatro tests de `max_int_seq` pasan (verificado). Observación adicional: en caso de empate (`max_a == max_b`) la función devuelve `b`; es una decisión de diseño razonable pero conviene documentarla.

4. Módulo `person.py`

Se responden las preguntas embebidas como `#TODO` en el módulo. Archivo de tests entregable: `test_person.py`. Las fechas se calculan relativas a *hoy* para que los tests sean deterministas.

4.1 `__init__` — ¿Es correcta la implementación?

Respuesta: funciona como simple contenedor de datos, pero **no valida** sus argumentos, por lo que *permite construir estados inválidos* (p. ej. una fecha de nacimiento en el futuro, o `studies` que no sea un `Studies`). Bajo la especificación “un `Person` no debe poder estar en un estado inválido”, la implementación es *incompleta*.

```

1  def test_init_guarda_atributos():          # PASA
2      p = Person(date(2000,1,1), "David", Studies.BACHELOR)
3      assert p.name == "David" and p.studies == Studies.BACHELOR
4
5  def test_init_no_valida_fecha_futura():    # FALLA (segun spec)
6      futuro = date(date.today().year + 5, 1, 1)
7      p = Person(futuro, "David")

```

```

8      assert p.birthdate <= date.today()      # el constructor debería
        rechazarla

```

Defecto (fault): ausencia de validación en el constructor. **Error (infección):** el objeto queda con `_birthdate` en el futuro (estado inválido). **Falla (failure):** luego `age` devuelve una edad negativa y `__str__` muestra datos absurdos.

Corrección sugerida: validar en el constructor.

```

1  def __init__(self, birth, name, studies=Studies.NONE):
2      if not isinstance(birth, date) or birth > date.today():
3          raise ValueError("birth debe ser una fecha no futura")
4      if not name:
5          raise ValueError("name no puede ser vacio")
6      if not isinstance(studies, Studies):
7          raise TypeError("studies debe ser un Studies")
8      self._birthdate = birth; self._name = name; self._studies =
        studies

```

4.2 ¿Sería correcto permitir modificar la fecha de nacimiento y el nombre?

Fecha de nacimiento: no. Es un dato inmutable en la realidad (nunca cambia); permitir modificarla sólo habilita estados inconsistentes. Por eso el módulo expone `birthdate` sólo como *property* de lectura, sin *setter*: es la decisión correcta.

Nombre: es discutible. Un nombre *puede* cambiar legalmente, así que un *setter* sería defendible. Pero si el nombre forma parte de la identidad del objeto (se usa en `__eq__` y en un eventual `__hash__`), hacerlo mutable rompe invariantes al usar el objeto en `set` / `dict`. Mantenerlo inmutable (como está) es la opción más segura.

4.3 `age` — ¿Es correcta la implementación?

Respuesta: no. Calcula la edad restando sólo los años y *ignora si el cumpleaños ya ocurrió* este año. Para quien nació más tarde en el año que la fecha actual, sobreestima la edad en 1.

```

1  def test_age_cumpleaños_ya_paso():          # PASA
2      p = Person(date(date.today().year - 30, 1, 1), "David") # nació 1-ene
3      assert p.age == 30
4
5  def test_age_cumpleaños_no_llego():         # FALLA (off-by-one)
6      p = Person(date(date.today().year - 30, 12, 31), "David") # nació
        31-dic
7      assert p.age == 29      # hasta diciembre todavía tiene 29

```

Defecto (fault): `years = today.year - birth.year`, sin ajustar por mes/día. **Error (infección):** para un nacido el 31-dic, `years` vale 30 cuando el valor correcto es 29. **Falla (failure):** `age` retorna un número mayor al real (verificado: devuelve 30 en vez de 29).

Corrección: restar 1 si el cumpleaños de este año aún no llegó.

```

1  today = date.today()
2  years = today.year - self._birthdate.year - (
3      (today.month, today.day) < (self._birthdate.month,
4          self._birthdate.day)
5  )
6  return years

```

4.4 `studies.setter` — ¿Es correcta la implementación?

Respuesta: no. El *setter* asigna directamente sin validar. Esto (a) acepta valores que no son `Studies`, rompiendo la invariante de tipo, y (b) permite *bajar* el nivel de estudios, lo cual no tiene sentido (no se “des-estudia” un título).

```

1  def test_studies_setter_actualiza():          # PASA
2      p = Person(date(2000,1,1), "David", Studies.NONE)
3      p.studies = Studies.MASTERS
4      assert p.studies == Studies.MASTERS
5
6  def test_studies_setter_valida_tipo():        # FALLA (sin validacion)
7      p = Person(date(2000,1,1), "David", Studies.MASTERS)
8      p.studies = "PhD"                        # valor invalido
9      assert isinstance(p.studies, Studies)
10
11 def test_studies_setter_no_regresa():          # FALLA (permite regresion)
12     p = Person(date(2000,1,1), "David", Studies.MASTERS)
13     p.studies = Studies.PRIMARY
14     assert p.studies == Studies.MASTERS

```

Defecto (fault): asignación sin validación (`self._studies = new_studies`). **Error (infección):** `_studies` queda con un `str` arbitrario o con un nivel menor al previo. **Falla (failure):** un `Person` con nivel `MASTERS` pasa a `PRIMARY`, o `_studies` deja de ser un `Studies` (verificado: ambas asignaciones se aceptan sin error).

Corrección: validar tipo e impedir regresión.

```

1 @studies.setter
2 def studies(self, new_studies):
3     if not isinstance(new_studies, Studies):
4         raise TypeError("studies debe ser un Studies")
5     if new_studies.value[1] < self._studies.value[1]:
6         raise ValueError("el nivel de estudios no puede disminuir")
7     self._studies = new_studies

```

4.5 `__eq__` — ¿Es correcta la implementación?

Respuesta: no, tiene dos defectos. (1) Compara los nombres con `is not` (identidad de objetos) en lugar de `!=` (igualdad de valor). (2) Compara `age` en vez de `_birthdate`, lo que hace que la igualdad dependa del día en que se ejecute y que dos personas de distinta fecha pero misma edad se consideren iguales. Además, al definir `__eq__` sin `__hash__`, la clase queda *no hashable*.

```

1 def test_eq_mismos_datos_nombre_interno(): # PASA (por interning)
2     a = Person(date(1990,5,5), "David", Studies.NONE)
3     b = Person(date(1990,5,5), "David", Studies.NONE)
4     assert a == b
5
6 def test_eq_nombre_no_interno(): # FALLA (por `is not`)
7     n1 = "".join(["D","a","v","i","d"]); n2 =
8         "".join(["D","a","v","i","d"])
9     a = Person(date(1990,5,5), n1, Studies.NONE)
10    b = Person(date(1990,5,5), n2, Studies.NONE)
11    assert a == b
12
13 def test_person_es_hashable(): # FALLA (falta __hash__)
14    a = Person(date(1990,5,5), "David", Studies.NONE)
15    assert hash(a) is not None

```

Defecto (fault): `if self._name is not other._name` usa identidad; y se compara `age` en vez de `_birthdate`. **Error (infección):** para dos nombres iguales pero contruidos en runtime (no *interned*), `is not` da `True` y la función retorna `False` prematuramente. **Falla (failure):** `a == b` devuelve `False` siendo lógicamente iguales (verificado). Y `hash(a)` lanza `TypeError: unhashable type`.

Corrección: comparar por valor, usar `_birthdate` y agregar `__hash__`.

```

1  def __eq__(self, other):
2      if not isinstance(other, Person):
3          return False
4      if self._name != other._name:          # FIX: != en lugar de
        `is not`
5          return False
6      if self._birthdate != other._birthdate: # FIX: comparar fecha,
        no age
7          return False
8      if self._studies != other._studies:
9          return False
10     return True
11
12  def __hash__(self):
13     return hash((self._name, self._birthdate, self._studies))

```

4.6 `max(a, b)` — Implementar y proveer tests

La consigna no fija qué significa “persona máxima”, así que se define un **orden explícito y documentado**: gana quien tiene *mayor nivel de estudios*; a igual nivel, gana quien tiene *mayor edad* (fecha de nacimiento más temprana); si empatan en todo, se devuelve `a`.

```

1  def max(a: Person, b: Person) -> Person:
2      """Persona maxima: mayor nivel de estudios; a igualdad, mayor edad."""
3      na, nb = a.studies.value[1], b.studies.value[1]    # nivel ISCED
        interno
4      if na != nb:
5          return a if na > nb else b
6      if a.birthdate != b.birthdate:
7          return a if a.birthdate < b.birthdate else b    # mas edad = fecha
        menor
8      return a

```

```

1  def test_max_por_estudios():          # PASA con la implementacion
2      a = Person(date(1990,1,1), "David", Studies.DOCTORATE)
3      b = Person(date(1990,1,1), "Isaac", Studies.BACHELOR)
4      assert max(a, b) is a
5
6  def test_max_desempata_por_edad():    # PASA con la implementacion
7      a = Person(date(1970,1,1), "David", Studies.MASTERS)    # mas grande
8      b = Person(date(2000,1,1), "Isaac", Studies.MASTERS)
9      assert max(a, b) is a

```

Criterio verificado ejecutando ambos tests. Si la cátedra prefiere otro orden (p. ej. sólo por edad), basta cambiar la clave de comparación; la estructura del código y de los tests se mantiene.

Resumen de defectos encontrados

Módulo / función	Defecto	Falla observable
<code>max_int_seq</code>	<code>== a</code> en vez de <code>== a[i]</code>	Con igual longitud siempre devuelve <code>b</code> .
<code>Person.__init__</code>	Sin validación de argumentos	Permite fecha futura / tipos inválidos.
<code>Person.age</code>	No ajusta por mes/día	Edad sobreestimada en 1 antes del cumpleaños.
<code>Person.studies</code> (setter)	Sin validación	Acepta tipos inválidos y permite bajar de nivel.
<code>Person.__eq__</code>	<code>is not</code> en vez de <code>!=</code> ; usa <code>age</code>	Personas iguales dan <code>!=</code> ; clase no hasheable.
<code>Person.max</code>	No implementada	Lanzaba <code>NotImplementedError</code> ; ya implementada.